ProcessMap Import, Document, and Text Analysis Specification

Version: 1.0 Status: Current implementation baseline Scope: Import wizard, file ingestion, text parsing, AI-assisted structuring/allocation, interpretation, and allocation pipeline Primary code owners:

- `src/components/TextImportModal.tsx`
- `src/components/CategoryBuilder.tsx`
- `src/components/StepAllocator.tsx`
- `src/utils/textParser.ts`
- `src/utils/importInterpretation.ts`
- `src/utils/claudeApi.ts`

---

1) Purpose and Objectives

The import subsystem converts unstructured or semi-structured source content into a `ProcessMapProject` with:

- hierarchical category/section/flow structure,
- semantic separation of process vs fact/context material,
- optional multi-file source grouping,
- operator review controls before final commit to project state.

Primary goals:

- reduce manual process-map authoring effort,
- preserve source signal quality and hierarchy,
- avoid silent data loss (unresolved items remain reviewable/unclassified),
- provide interpretable confidence signals for review.

---

2) Supported Inputs and Source Modes

2.1 Accepted source types

- Direct pasted text
- `.txt`, `.md`, `.text`
- `.docx` (via `mammoth`)
- `.pdf` (via `pdfjs-dist`)

2.2 Multi-file ingestion

- File picker supports `multiple`.
- For multi-file mode, each file is parsed into a dedicated `ParsedSourceFile` and merged for step review.
- Source metadata (`sourceId`, `sourceName`) is attached to parsed steps.

2.3 PDF page fidelity

- PDF parsing extracts text per page.
- Import interpretation operates on true page arrays (`sourcePageTexts`) rather than synthetic blank-line splitting.

---

3) Import Wizard UX Contract

Wizard steps:

- `paste` (Step 1: Paste or Upload)
- `confirm-ai` (AI send confirmation)
- `review` (Step 2: Review & Edit Structure)
- `categories` (Step 3: Define Categories)
- `allocate` (Step 4: Allocate Steps)

Key defaults:

- Smart Parse (AI) default ON (`useAi = true`)
- Grouping mode default `per_file`
- Dedupe policy default `within_file`

Step 2 actions:

- Continue to Customize Categories
- Quick Import

Notes:

- Step 4 receives full displayed hierarchy (`displayedSteps`) and not a flattened leaf-only list.
- This is required for subprocess container participation in AI auto-allocation.

---

4) Data Contracts

4.1 ParsedStep

`ParsedStep` fields:

- `id` , `label` , `description` , `nodeType` , `children` , `indent`
- semantic: `semanticKind` , `semanticScore` , `semanticSignals`
- source: `sourceId` , `sourceName`

4.2 Allocation contracts

`AllocatedCategory` :

- `name` , optional `kind` (`process` | `facts`)
- optional `sourceId` , `sourceName`
- `steps: ParsedStep[]`

`AllocatedToProjectOptions` :

- `groupBySource?: boolean`
- `dedupePolicy?: 'within_file' | 'cross_file_safe'`

4.3 Interpretation contracts

`ImportInterpretation` :

- `pages: PageInterpretation[]`
- per-cluster counts + review counts
- `reviewCount` , `totalBlocks`
- preview lists (overview/facts/unclassified labels)

---

5) Parsing and Cleanup Pipeline

Parsing entrypoint:

- `parseTextToSteps(text: string): ParsedStep[]`

5.1 Cleanup stages

- Remove ASCII-art / visual noise
- Remove separators/page markers/box lines
- Prompt-template stripping (if pasted prompt wrappers exist)
- Whitespace normalization and junk-line filtering

5.2 Structural inference

- Header detection:
    - numbered headers
    - markdown headers
    - uppercase section-like lines
- Header levels:
    - level 1 = category
    - level 2 = section
- Body lines become operational/process candidates under current section/category.

5.3 Type assignment

- Explicit `[type]` tags mapped first
- Fallback content classification:
    - decision cues,
    - action verb cues.

5.4 Semantic annotation

- `scoreFactStatement` computes fact-likelihood from:
    - cue starters,
    - definition patterns,
    - metric/baseline patterns,
    - penalties for decision/action signatures.
- Annotated as `semanticKind = fact|process`.

5.5 Normalization

- Duplicate merge with similarity safeguards (`shouldMergeDuplicate`)
- Overflow subgrouping for large child lists
- Misc grouping for too many tiny groups

---

6) AI-Assisted Parsing and Allocation

6.1 Smart Parse

- `smartParse(text)` calls Claude with strict formatting prompt.
- Expected output: numbered categories + tagged bullet subitems.
- Result re-enters local parser (`parseTextToSteps`).

6.2 AI allocation API

- `smartAllocateSteps(categories, steps)` returns category suggestions:
    - `id` , `category` , `confidence` , `reason`
- Response must be JSON and category-exact.
- Invalid allocations are filtered out.

6.3 StepAllocator dual-pass model

Implemented in `StepAllocator`:

Pass A (parents/subprocess containers):

- candidate set = nodes with children OR subprocess type.
- thresholds:
  - auto-accept if `confidence ⩾ 0.75`
  - review queue if `0.5 ⩽ confidence < 0.75`

Pass B (leaf steps):

- candidate set = non-subprocess leaves.
- thresholds:
  - auto-accept if `confidence ⩾ 0.8`
  - review queue if `0.5 ⩽ confidence < 0.8`
- fallback inheritance:
  - if leaf confidence is weak and parent assignment exists, inherit parent category.

Review UX:

- Split counters and queues:
  - `parent review`
  - `step review`
- Button copy:
  - `AI Auto-allocate (Subprocess + Steps)`

---

7) Category Policy and Facts Handling

Step 3 (`CategoryBuilder`) policy toggles:

- Include Facts & Context category
- Allow AI to propose new categories (UI placeholder for future behavior)

Facts policy behavior:

- Facts category is auto-injected when:
  - facts detected, or
  - explicit include-facts policy enabled.
- In allocator:
  - fact candidates are visually marked,
  - facts category supports dedicated drop guidance.

---

8) Interpretation and Review Intelligence (Step 2)

8.1 Interpretation model

`buildImportInterpretation(sourceName, pageTexts)`:

- builds page-level block classification across clusters:
  - context_process_category
  - process
  - subprocess
  - fact
  - unclassified
- computes confidence and needs-review metrics.

Needs-review criteria:

- block confidence < 0.75 OR cluster = unclassified.

8.2 Interpretation UI

Step 2 includes:

- cluster chips with counts,
- page overview toggle,
- filters:
  - All
  - Needs review
  - Unclassified
  - Facts
- summary banner (`reviewCount/totalBlocks`).

8.3 External intelligence artifacts

- Export content map HTML (`buildContentMapHtml`).
- Copy deep visual Claude prompt (`buildDeepVisualPrompt`) for PDF source.

---

9) Multi-file Grouping and Dedupe Semantics

9.1 Grouping mode

- `per_file` :
  - output project overview grouped by source file.
- `shared_categories` :
  - categories merged across sources.

9.2 Dedupe policy

- `within_file` : dedupe scope constrained per file.
- `cross_file_safe` : allows cross-file safe dedupe strategy.

9.3 Source-level review

- Step 2 shows per-file tabs for parsed file review.
- edits apply to active source context.

---

10) Import-to-Project Build Strategies

Two finalization paths:

10.1 Quick Import

- `stepsToProject(parsedSteps, name, isDraft, skipClustering?)`
- Direct parse-to-map conversion.

10.2 Custom category allocation import

- `allocatedToProject(allocatedCats, graveyard+unallocated, name, isDraft, options)`
- Honors category/facts/source policies.

Safety invariant:

- unresolved/unallocated items are not silently dropped.
- they are merged into review buckets (graveyard/unclassified map path).

---

11) Import Architecture Sequence

```
User input/file(s)
  → TextImportModal
      → parseSingleFile (pdf/docx/text)
      → (optional) smartParse (Claude)
      → parseTextToSteps
      → Step 2 review + interpretation
      → Step 3 CategoryBuilder policy
      → Step 4 StepAllocator dual-pass AI allocation
      → allocatedToProject OR stepsToProject
      → useAppStore.importProject(json)
```

---

12) Error Handling and Operational Constraints

Error pathways:

- AI parse failure:
    - surface message and return to paste step.
- File import failure:
    - fallback error text in source textarea.
- Invalid AI allocation JSON:
    - throw allocation error and preserve current unresolved state.

Operational constraints:

- Claude key in local storage (`processmap-claude-api-key`).
- Browser direct API invocation enabled by Anthropic browser-access header.

---

13) Security and Privacy Considerations

- Smart Parse and allocation send user-provided text to Anthropic.
- UI includes explicit confirmation and token/cost estimate before AI send.
- No backend proxy exists; API key is browser-local.
- Operators should not import sensitive data without policy review.

---

14) Extension Points and Roadmap Hooks

Designed extension points:

- category expansion from AI suggestions (currently UI placeholder).
- richer cross-file dedupe heuristics by policy tier.
- confidence threshold tuning per corpus profile.
- stronger hierarchy-to-layout preservation and auto-link inference.

Recommended future hardening:

- schema validation for parsed AI outputs using typed guards.
- import telemetry for confidence and correction feedback loops.
- optional backend AI proxy with tenant-level key isolation.

15) Acceptance Criteria Snapshot

Functional:

- parse text/file input to hierarchical steps.
- support AI parse with confirmation step.
- support custom categories + allocation.
- include subprocesses in AI allocation pass.
- preserve unresolved items into final project.
- support interpretation filters and external artifact actions.

UX:

- 4-step wizard visible with clear step titles.
- review-by-exception allocation with parent/leaf split.
- operator can complete end-to-end import without editing raw JSON.

Quality:

- build and lint pass.
- no silent drop of unresolved content.
- deterministic output for identical non-AI input pipelines.